Practical Malware Analysis

Chapter-by-Chapter Explained Summary

Michael Sikorski & Andrew Honig | No Starch Press

How to read this summary. The book teaches you to dissect unknown Windows programs, moving from easy triage to deep reverse engineering. Each chapter below has a one-line gist, a plain explanation of what it teaches, the key points to remember, and the tools and terms you will meet. The whole book is built on the idea that you start with the cheapest technique and escalate only when you must.

Introduction

Chapter 1 – Malware Analysis Primer

Defines malware, the goals of analysis, the four techniques, and the ground rules.

What it explains

Malware is any software that harms a user, computer or network. The purpose of analysis is usually to support a response to an intrusion: what happened, which machines and files are affected, and how to detect and contain it. The chapter introduces the output you are aiming for: **host-based indicators** (files created, registry keys changed, mutexes) and **network signatures**. Unlike antivirus signatures, host indicators describe what malware *does*, so they still work when the file changes form or has been deleted. It then lays out the two families of analysis, static (examine without running) and dynamic (run it), each split into basic and advanced.

Key points

- **Basic static:** inspect the file without reading instructions. Fast but easily defeated.
- **Basic dynamic:** run it in a safe lab and observe behavior.
- **Advanced static:** disassemble and read the code. **Advanced dynamic:** use a debugger on the running program.
- **Malware families:** backdoor, botnet, downloader, launcher, information stealer, rootkit, scareware, spam sender, worm/virus. Many samples mix several.
- **Mass vs. targeted:** targeted malware is custom-built and is the real danger, because security products rarely know it.
- **Three rules:** do not drown in details, use varied tools and switch when stuck, and expect a cat-and-mouse game as authors counter each technique.

Tools & terms: disassembler, debugger, host-based indicator, network signature, backdoor, launcher, rootkit

PART I – BASIC ANALYSIS (Chapters 2–4)

Chapter 2 – Basic Static Techniques

Learn as much as possible from a file without ever running it.

What it explains

Static analysis is the first step. You start by scanning the file with several antivirus engines, since each uses different signatures. This helps but is not decisive: authors easily change their code, and rare samples are simply not in the databases. Next you compute a **hash** (such as MD5 or SHA-1), a unique fingerprint used to label the file, share it with others and search for earlier reports. Then you extract **strings**, readable text that can reveal URLs, file names, registry keys, error messages and commands. Finally you read the **PE file format**, the structure of Windows executables, where the import table shows which Windows functions the program asks for.

Key points

- **Packing and obfuscation:** a packed program shows few readable strings and very few imports. PEiD can identify common packers (such as UPX).
- **Linking:** static, runtime and dynamic linking. Dynamic imports are most common and most informative; runtime linking (LoadLibrary/GetProcAddress) is used to hide imports.
- **Imports reveal capability:** the book contrasts an unpacked keylogger-like sample (hooking, key-mapping and file functions) with a packed program that imports almost nothing, a dead end for static analysis.
- **PE sections:** .text (code), .rdata (imports and read-only data), .data (globals), .rsrc (resources, often hiding embedded payloads). A section with raw size 0 but nonzero virtual size suggests packing.
- **PE header details:** compile timestamp, imports/exports, resources. Useful clues but can be forged.

Tools & terms: antivirus/VirusTotal, MD5/SHA-1, Strings, PEiD, Dependency Walker, PEview, Resource Hacker, UPX

Chapter 3 – Malware Analysis in Virtual Machines

Build a safe lab so you can run malware without harming real systems.

What it explains

Running unknown malware on a normal machine is dangerous. You can use dedicated physical machines on an air-gapped network, but they are slow to restore and have no Internet, which many samples need. So analysts typically use **virtual machines**: a guest OS running inside a host, isolated from it. The chapter walks through installing a Windows guest in VMware, installing VMware Tools, and then configuring networking so the malware can talk, but only to something harmless.

Key points

- **Networking choices:** disconnected (safest but you see no network behavior), **host-only** (private LAN between host and guest, no Internet), or a custom LAN of several VMs where one runs the malware and another supplies fake services.
- **Connecting to the Internet** is sometimes needed but must be done with care and awareness of the risk.
- **Snapshots:** save a clean baseline and revert after each run. This is the most important feature for analysts.
- **Peripherals and file transfer:** disconnect USB devices and be careful with drag-and-drop and shared folders, which can become a path for malware to reach the host.

- **Risks:** malware may detect the VM and change behavior, and VM software itself can have vulnerabilities, so keep the host patched and firewalled.

Tools & terms: VMware, host-only network, snapshot, air gap, VMware Tools

Chapter 4 – Basic Dynamic Analysis

Run the malware in the lab and watch what it does.

What it explains

Dynamic analysis comes after basic static analysis, because running malware is riskier. It shows true behavior: a string in the file does not prove an action actually happens. The quickest option is an automated **sandbox** that runs the file and reports its behavior. Sandboxes are convenient but have drawbacks (they may not supply required command-line arguments, may not run DLLs properly, may be detected, and only show one execution path). So analysts also run the sample themselves while watching the system and the network.

Key points

- **Running code:** executables run directly, while DLLs can be launched through rundll32.exe. Services must be installed first.
- **Process Monitor (Procmon):** records file, registry, process and network activity. Filtering is essential because the output is huge.
- **Process Explorer:** shows running processes, handles and DLLs, and a verify option checks signatures. Comparing strings in memory against those on disk reveals disguised or replaced processes.
- **Regshot:** snapshots the registry before and after to show what changed.
- **Network simulation:** ApateDNS fakes DNS answers to redirect traffic, Netcat listens to see what the malware sends, and Wireshark captures packets. INetSim simulates common services.
- **Also covered:** analyzing malicious documents and the limits of dynamic analysis, because not every code path executes.

Tools & terms: Procmon, Process Explorer, Regshot, ApateDNS, Netcat, Wireshark, INetSim, rundll32

PART II – ADVANCED STATIC ANALYSIS (Chapters 5–8)

Chapter 5 – A Crash Course in x86 Disassembly

The assembly-language foundation everything else rests on.

What it explains

Basic techniques are like examining the outside of a body in an autopsy: useful, but you need more for the whole story. Disassembly gives it. The chapter starts with levels of abstraction: authors write in high-level languages, a compiler produces machine code, and analysts work backwards from **assembly**, the human-readable form of machine code. It then covers how the CPU and memory work and the instructions you will see most.

Key points

- **Instruction anatomy:** mnemonic plus operands, encoded as opcodes. x86 stores multi-byte values in little-endian order.
- **Registers:** general-purpose (EAX, EBX, ECX, EDX, ESI, EDI, EBP, ESP), segment registers, the EFLAGS register (flags such as zero and carry) and EIP, the instruction pointer.
- **Common instructions:** mov, lea, add, sub, inc/dec, mul/div, logical operations (and, or, xor, shifts), nop.

- **The stack:** push/pop, call/ret, stack frames and local variables, with ESP and EBP tracking the current frame.
- **Conditionals and branching:** cmp and test set flags, then jumps (jz, jnz, jmp and others) change flow.
- **Rep instructions** for string and block operations, and how C's main function receives arguments.

Tools & terms: register, EFLAGS, stack frame, opcode, little-endian, jmp/call/ret

Chapter 6 – IDA Pro

Learn the disassembler most analysts live in.

What it explains

IDA Pro is an interactive disassembler. It disassembles the whole program, finds functions, identifies local variables and recognizes library code (FLIRT), then lets you refine everything. You load a file, choose a view, and work by navigating, renaming and annotating. Your progress is saved in a database (idb) you can reopen later, which makes IDA a notebook as much as a viewer.

Key points

- **Loading and views:** graph mode (shows flow as boxes and arrows) and text mode, plus helpful windows (functions, strings, imports, exports, names).
- **Navigation:** jump to an address, search, and use the navigation band.
- **Cross-references:** code xrefs show who calls a function; data xrefs show who uses a variable or string. These are the fastest way to find interesting behavior, for example every function that calls a networking API.
- **Making the code readable:** rename locations, add comments, format operands, apply named constants, redefine code and data.
- **Extending IDA:** IDC scripts, IDAPython, and plug-ins (the Hex-Rays decompiler and BinDiff are commercial examples).

Tools & terms: idb, FLIRT, cross-reference, graph view, IDC, IDAPython

Chapter 7 – Recognizing C Code Constructs in Assembly

Read groups of instructions as loops, ifs and structures, not one by one.

What it explains

Programs are millions of instructions, so analysts cannot study each. Malware is usually written in C, so you learn how common **code constructs** look once compiled and then recognize them at a glance. The learning direction is helpful: you start from disassembly and recover the high-level idea.

Key points

- **Variables and arithmetic:** global vs. local variables and how they are addressed.
- **Branches:** if/else and nested ifs, recognized from compare-and-jump patterns.
- **Loops:** for and while loops have a characteristic initialize, compare, body and increment shape.
- **Functions:** calling conventions (cdecl, stdcall, fastcall) define how arguments are passed and cleaned up.
- **Switch statements:** compiled either as an if-style chain or as a jump table.
- **Arrays, structs, linked lists** and how compilers differ (Visual Studio vs. GCC) in the code they emit.

Tools & terms: code construct, calling convention, jump table, stack frame, compiler differences

Chapter 8 – Analyzing Malicious Windows Programs

The Windows concepts malware relies on, and why they leave indicators.

What it explains

Most malware targets Windows and works closely with it. Understanding the Windows API explains behavior, helps you create host-based indicators and shows how malware runs code without an obvious jump or call. Malware is often poorly formed and does unexpected things, so even experienced Windows programmers benefit from this chapter.

Key points

- **API conventions:** types, Hungarian notation and handles.
- **File system:** CreateFile, ReadFile, WriteFile, file mapping, special files and alternate data streams (hidden data attached to files).
- **Registry:** root keys, keys that run programs automatically (a common persistence spot), common functions, and .reg scripting.
- **Networking:** Berkeley-compatible sockets (client vs. server side) and the higher-level WinINet API.
- **Execution:** DLLs (DllMain), processes (CreateProcess), threads (CreateThread), mutexes (used to ensure only one copy runs, and a classic indicator), and services (code that runs in the background, started by the service manager).
- **Also:** the Component Object Model (COM), exceptions and structured exception handling, and kernel vs. user mode, with the native API sitting beneath the Windows API.

Tools & terms: Windows API, handle, mutex, service, DLL, COM, registry Run keys, kernel mode

PART III – ADVANCED DYNAMIC ANALYSIS (Chapters 9–11)

Chapter 9 – Debugging

How debuggers let you watch and control a running program.

What it explains

A disassembler gives a snapshot of a program before its first instruction runs. A debugger gives a live view: you see memory, registers and function arguments change, and you can alter them. This chapter covers concepts common to all debuggers, before the next two chapters introduce specific ones.

Key points

- **Source-level vs. assembly-level:** developers use source-level debuggers; analysts have no source, so they use assembly-level ones.
- **Stepping:** single-step runs one instruction; step-over runs a whole call as one step, step-into follows it.
- **Breakpoints:** pause execution at chosen places (software, hardware, conditional and memory variants).
- **Exceptions:** first-chance vs. second-chance handling and common exceptions; malware sometimes relies on them.
- **Modifying execution:** change a register, flag, variable or instruction pointer to force a path, for example to make a check succeed.

Tools & terms: breakpoint, step-over/step-into, exception, user mode vs. kernel mode

Chapter 10 – OllyDbg

The popular free user-mode debugger for 32-bit malware.

What it explains

OllyDbg is easy to use and extensible. Its fork, Immunity Debugger, adds a Python interpreter. You load malware by opening the executable or by attaching to a running process, then use its panes (disassembly, registers, stack, memory dump) to control and inspect execution.

Key points

- **Running and stepping:** run, pause, step-into (F7) and step-over (F8).
- **Breakpoints:** software, conditional, hardware and memory breakpoints, each suited to different tasks.
- **Tracing:** standard back trace, call stack and run trace. The book demonstrates tracing a Poison Ivy sample.
- **Other features:** memory map, handles, exception handling, patching code, analyzing shellcode, and assisted features such as bookmarks and a command line.
- **Plug-ins:** OllyDump (dump unpacked code) and hide-debugger plug-ins, plus scriptable debugging in Immunity via Python.

Tools & terms: OllyDbg, ImmDbg, software/hardware/memory breakpoint, run trace, OllyDump

Chapter 11 – Kernel Debugging with WinDbg

Analyze drivers and rootkits that run inside the OS kernel.

What it explains

WinDbg is Microsoft's free debugger, and its key advantage for analysts is kernel debugging. Kernel code is hard to study because drivers load into memory, stay resident, and are reached indirectly through device objects instead of direct calls. Malware authors use kernel mode for extra power and stealth, typically in rootkits.

Key points

- **Drivers and devices:** drivers handle requests sent to device objects; malware drivers create their own devices for user-mode components to talk to.
- **Setup:** kernel debugging needs two machines (typically two VMs) linked virtually.
- **Using WinDbg:** reading memory, arithmetic operators, setting breakpoints, searching symbols, viewing structures, and configuring Microsoft symbols.
- **Practical work:** look at user-space and kernel-mode code, find driver objects, and analyze a real rootkit that hooks system functions to hide itself.
- **Also:** interrupts and the differences on newer Windows versions.

Tools & terms: WinDbg, driver object, device object, rootkit, symbols, kernel mode

PART IV – MALWARE FUNCTIONALITY (Chapters 12–15)

Chapter 12 – Malware Behavior

A tour of what malware commonly does, so you recognize it quickly.

What it explains

This chapter gives a well-rounded catalog of behaviors so you can form good hypotheses and confirm them. It cannot cover everything, but it covers the main patterns.

Key points

- **Downloaders and launchers:** a downloader fetches and runs more malware (often URLDownloadToFile then WinExec); a launcher installs malware covertly.
- **Backdoors:** remote access, often over HTTP on port 80. A **reverse shell** makes the victim connect out. **RATs** are controlled by an operator; **botnets** take identical orders from a central server.
- **Credential stealing:** hash dumping, GINA interception (hooking the Windows logon), and keylogging by hooking (SetWindowsHookEx) or polling (GetAsyncKeyState).
- **Persistence:** Run keys, trojanized system binaries, and DLL load-order hijacking, where a malicious DLL is loaded in place of a legitimate one.
- **Privilege escalation:** for example enabling SeDebugPrivilege.
- **Hiding:** user-mode rootkits using IAT hooking or inline hooking to alter what programs see.

Tools & terms: reverse shell, RAT, botnet, keylogger, GINA, DLL hijacking, SeDebugPrivilege, IAT/inline hook

Chapter 13 – Covert Malware Launching

How malware runs code inside other processes to blend in.

What it explains

As users learned to check Task Manager, authors found ways to hide. Launching code covertly means making it run without a visible, suspicious process of its own. You learn the API patterns that give each technique away.

Key points

- **Launchers:** often hold the real payload in their resource section and extract it using FindResource, LoadResource and SizeofResource.
- **DLL injection:** force a target process to load a malicious DLL.
- **Direct injection:** write raw code into a process and run it there.
- **Process replacement:** start a legitimate process suspended and swap in malicious code.
- **Hook injection:** use SetWindowsHookEx (local or remote hooks, thread targeting) to get a DLL loaded in other processes; common in keyloggers.
- **Detours and APC injection:** modify existing code in place, or queue code to run when a thread enters an alertable state.

Tools & terms: process injection, CreateRemoteThread, WriteProcessMemory, SetWindowsHookEx, detours, APC

Chapter 14 – Data Encoding

Finding and decoding the encoding malware uses to hide its data.

What it explains

Malware encodes data to mask intent: network traffic, configuration (such as command-and-control addresses), stolen data in staging files, and internal strings. Your job has two parts: **identify the encoding function**, then use that knowledge to **decode** the data.

Key points

- **Simple ciphers:** Caesar, single-byte XOR (including variants that preserve NULL bytes), and other basic schemes such as Base64.
- **Recognizing them:** look for XOR in loops and search for characteristic constants and alphabets.
- **Standard cryptography:** spot strings and imports, search for cryptographic constants (tools like Krypto ANALyzer), and look for high-entropy content.
- **Custom encoding:** authors modify standard methods or combine them to defeat detection.
- **Decoding strategies:** use a self-decoding approach (let the malware decode it), write your own decoding function, or use instrumentation for generic decryption.

Tools & terms: XOR, Base64, entropy, cryptographic constants, KANAL, self-decoding

Chapter 15 – Malware-Focused Network Signatures

Build network detection that outperforms rules made from captured traffic alone.

What it explains

Network defenses (firewalls, DNS sinkholes, proxies, IDS/IPS) use IPs, ports, domains and traffic content. Because those basics are widely supported, they are the first things analysts investigate. The chapter's central claim is that rules written after understanding the code are more precise and give fewer false positives than rules based only on what you saw on the wire.

Key points

- **Indicators:** IP addresses, domains, URLs and HTTP details; indirection tactics make some of these unreliable.
- **Know the code behind the traffic:** find the networking code (WinSock, WinINet or COM), see which content is hard-coded and which is ephemeral, and identify encoding steps.
- **Writing signatures:** for example with Snort, targeting multiple elements and the stable parts of the protocol.
- **Do not overanalyze:** decide how deep you need to go; the goal is a useful signature, not a complete understanding of every byte.
- **Hiding in plain sight:** malware may mimic legitimate traffic, so look for subtle differences.

Tools & terms: Snort, sinkhole, beaconing, User-Agent, WinINet, hard-coded vs. ephemeral data

PART V – ANTI-REVERSE-ENGINEERING (Chapters 16–19)

Chapter 16 – Anti-Disassembly

Code built to make your disassembler lie.

What it explains

Any code that runs can be reverse-engineered, but authors raise the cost by crafting code or data that makes tools show a wrong listing. This also fools automated systems that rely on disassembly. The chapter first explains the two disassembly algorithms, then the tricks that exploit them.

Key points

- **Linear vs. flow-oriented disassembly:** linear reads bytes in order, while flow-oriented follows jumps (as IDA does); each can be tricked.
- **Tricks:** jump instructions with the same target, a jump with a constant condition, and impossible disassembly, where the same bytes are meaningful as two overlapping instructions.
- **Hiding flow control:** function pointers, return-pointer abuse and misuse of structured exception handlers.
- **Defenses:** fix the listing by hand, NOP out bytes, use IDA scripts or hotkeys, and add missing cross-references.

Tools & terms: linear/flow-oriented disassembly, opaque predicate, return pointer abuse, SEH

Chapter 17 – Anti-Debugging

Detecting or disrupting the analyst's debugger.

What it explains

Once malware realizes it is being debugged, it may change its behavior or crash. There are hundreds of techniques, and the book teaches the most common and the skill of defeating new ones.

Key points

- **API checks:** functions such as IsDebuggerPresent and CheckRemoteDebuggerPresent.
- **Manual structure checks:** flags in the Process Environment Block, such as BeingDebugged and process-heap flags.
- **System residue:** searching for debugger files, registry keys or windows.
- **Detecting behavior:** INT scanning (finding breakpoint bytes), code checksums and timing checks.
- **Interfering:** TLS callbacks that run before the main code, exceptions (INT 3, INT 2D), and PE-header or OutputDebugString vulnerabilities in debuggers.
- **Defenses:** patch the check, change the return value, or use hide-debugger plug-ins.

Tools & terms: PEB, IsDebuggerPresent, TLS callback, timing check, INT 3, INT 2D

Chapter 18 – Anti-Virtual Machine Techniques

Malware that tries to tell it is running in a VM.

What it explains

If the malware detects a VM, it may behave differently or refuse to run. It is most common in widely deployed malware (bots, scareware, spyware), because honeypots often use VMs. The trend has declined as ordinary users and administrators now run VMs too. The chapter focuses on VMware.

Key points

- **Artifacts:** VMware leaves traces such as processes, services, registry entries and network adapter addresses.
- **Bypassing:** remove VMware Tools and similar software, or adjust VM configuration settings.
- **Memory and instruction tricks:** checking descriptor-table registers (the Red Pill and No Pill techniques), instructions like str, and querying the VMware I/O communication port.
- **Detecting it in your analysis:** highlight suspicious anti-VM instructions in IDA, use tools such as ScoopyNG, then patch the checks.

Tools & terms: VMware artifacts, Red Pill, No Pill, I/O port, ScoopyNG

Chapter 19 – Packers and Unpacking

How packers hide programs, and a step-by-step way to undo them.

What it explains

Packers compress or encrypt an executable and add an **unpacking stub** that restores it at runtime. Packed files hide from antivirus, resist static analysis and are smaller. Most packers are free and easy, so packed malware is extremely common.

Key points

- **Anatomy:** the stub unpacks the original program, resolves its imports and then makes the **tail jump** to the **original entry point (OEP)**.
- **Recognizing packed files:** very few imports, abnormal section sizes, high entropy, packer detectors (PEiD) and tools like Mandiant Red Curtain.
- **Unpacking options:** automated static unpacking, automated dynamic unpacking, or manual dynamic unpacking in a debugger.
- **Manual process:** find the OEP, dump the process from memory (for example with OllyDump) and rebuild the import table (for example with Import Reconstructor, or by hand).
- **Goal:** you only need a version that behaves the same, not a perfect copy of the original. Common packers mentioned include UPX, PECompact, ASPack, Petite, WinUpack and Themida.

Tools & terms: packer, unpacking stub, tail jump, OEP, entropy, import reconstruction

PART VI – SPECIAL TOPICS (Chapters 20–22)

Chapter 20 – Shellcode Analysis

Analyzing raw, self-contained executable code.

What it explains

Shellcode is a payload of raw code, often delivered with an exploit or used for process injection. It cannot rely on the Windows loader, so its author must handle placement, relocation and library loading manually. The chapter explains these tricks with real examples.

Key points

- **Loading for analysis:** use a helper such as shellcode_launcher.exe, since shellcode is not a normal executable file.
- **Position-independent code:** it must find its own address, using call/pop or fnstenv.
- **Manual symbol resolution:** find kernel32.dll in memory by walking process structures, then locate functions by parsing export tables, often using **hashed export names**.
- **Tips:** look for common opcodes (calls, jumps, loops) when hunting for shellcode inside files.

Tools & terms: shellcode, position-independent code, call/pop, fnstenv, PEB, hashed exports

Chapter 21 – C++ Analysis

How object-oriented code looks after compilation.

What it explains

Malware analysis happens without source, but the source language shapes the assembly. C++ adds features that confuse analysis if you do not know their compiled forms.

Key points

- **Classes and objects:** like structs that also carry function information.
- **The this pointer:** every method gets the object pointer as a hidden argument.
- **Overloading and name mangling:** compilers encode types in function names; IDA can decode them.
- **Virtual functions and vtables:** an object holds a pointer to a table of function addresses; calls become indirect calls through that table, a pattern you learn to spot.
- **Creating and destroying objects:** constructors, destructors and the new operator.

Tools & terms: this pointer, vtable, name mangling, virtual function, constructor

Chapter 22 – 64-Bit Malware

Differences you need to know when malware is x64.

What it explains

Almost all malware at the time was 32-bit, but 64-bit samples exist, mainly because code that must run inside a 64-bit process, or a 64-bit kernel driver, must itself be 64-bit. The x64 instruction set is close to x86, so much is familiar, but some changes matter.

Key points

- **Why 64-bit:** to interact with 64-bit processes and kernels, and sometimes to evade tools that lack x64 support.
- **Calling convention:** the first four arguments go in registers rather than on the stack.

- **Registers and addressing:** wider registers, more registers and instruction-pointer-relative addressing.
- **Tooling:** at the time, OllyDbg did not support x64, WinDbg did, and IDA needed its Advanced version.

Tools & terms: x64, calling convention, WinDbg, IDA Advanced

APPENDICES

- **Appendix A – Important Windows Functions:** a reference of Windows API functions commonly seen in malware, with a short explanation of each.
- **Appendix B – Tools for Malware Analysis:** a list of useful tools, with their purposes.
- **Appendix C – Solutions to Labs:** short answers plus a detailed walkthrough for each lab, so you can check your work or learn from the full analysis.

The big picture

Basic static, then basic dynamic, then advanced static and dynamic: start cheap, escalate only as needed. Build a safe lab, learn to read assembly as constructs, track Windows API behavior to create indicators, and expect anti-analysis tricks (anti-disassembly, anti-debugging, anti-VM, packing) that you must learn to defeat. The end product is usually a set of host and network signatures plus a clear account of what the malware does.

Note: the book dates from around 2012, so its examples assume Windows XP-era labs and tools like IDA Pro and OllyDbg. The principles still hold; modern practice often substitutes Ghidra and x64dbg. This note comes from general background knowledge, not the book.